What `TargetKeygenSamplerCallers.ec` Proves

A guide to the HAETAE key-generation sampler-caller refinement

HAETAE reference EasyCrypt artifact

Artifact state checked 2026-07-13

Abstract

The file `easycrypt/refinement/TargetKeygenSamplerCallers.ec` proves a deliberately narrow refinement result for three Jasmin-extracted HAETAE key-generation callers. Each generated caller is relationally equivalent to an authored EasyCrypt procedure whose loop counter and call schedule are written as integer formulas. Both sides still invoke the *same extracted sampler leaf*. The proof obtains final-result equality by coupling corresponding loop iterations and equal leaf-call arguments; the exported postcondition is only equality of the returned result, not an explicit call trace. The result does not validate SHAKE or sampling semantics. This guide explains the six lemmas in the refinement file, the invariants that make the three main proofs work, and the boundary between this checked result and the remaining key-generation proof obligations.

Short answer. `easycrypt/refinement/TargetKeygenSamplerCallers.ec` shows that, from equal inputs, the extracted matrix, vector, and eta callers return equal arrays to authored schedule programs. The proof establishes this by aligning corresponding calls to the same extracted leaves; the theorem does not expose a trace-equality postcondition. It is a schedule-oriented result-equivalence, not a sampler-correctness theorem.

Contents

1 Where the File Fits

The checked path has four distinct artifacts. Keeping their roles separate is essential to interpreting the theorem correctly.

`haetae-ref-jasmin/jasmin/keypair.jazz:841--972`

↓ reproducible `jasmin2ec` extraction

`easycrypt/extract/keygen-sampler-callers/KeygenSamplerCallersTarget.ec`

↕ pRHL equivalences in `easycrypt/refinement/TargetKeygenSamplerCallers.ec`

`easycrypt/spec/KeygenSamplerCallersSpec.ec` (`CallerSpec`)

Artifact	Responsibility
Generated target	The value-array EasyCrypt model extracted from the selected Jasmin caller procedures and their leaf closure. Generated files are not edited by hand.
Authored specification	Integer schedule functions, their word encodings, supporting arithmetic lemmas, mode-2 facts, and three simple caller programs.
Authored refinement	Three leaf self-equivalences and three relational proofs between generated callers and authored callers.
Verification gate	Source fingerprinting, extraction-drift detection, `-no-eco` compilation, and scans for proof holes and new axioms.

The proof manifest compiles these theories in dependency order:

1. `easycrypt/extract/keygen-sampler-callers/KeygenSamplerCallersTarget.ec`;
2. `easycrypt/spec/KeygenSamplerCallersSpec.ec`;
3. `easycrypt/refinement/TargetKeygenSamplerCallers.ec`.

2 The Schedule Being Compared

The authored module `KeygenSamplerCallersSpec.CallerSpec` replaces the generated `W64` loop variables with mathematical integer counters. It then converts each integer schedule back to a `W64` leaf argument. This normal form makes the intended base and nonce formulas visible without changing the leaf implementation. In the formulas below, $\mathrm{uint}(x)$ abbreviates `W64.to_uint(x)`.

The generated procedures compute the same values incrementally. For example, the vector target initializes

$$nonce \leftarrow (k \ll 8) + m, \qquad base \leftarrow 0,$$

then increments `nonce` by 1 and `base` by 256 after each call. The authored procedure instead recomputes

$$\mathrm{vector\_nonce\_word}(\mathrm{uint}(k), \mathrm{uint}(m), i) \quad \text{and} \quad \mathrm{linear\_base\_word}(i).$$

The refinement proof establishes that these incremental and closed-form views remain equal at every iteration.

Caller	Iteration space	Leaf arguments at an iteration	Extracted leaf
Matrix A	$0 \le i < \mathrm{uint}(rows)$, $0 \le j < \mathrm{uint}(cols)$	seed offset 0; base $256(i\,\mathrm{uint}(cols) + j)$; nonce $256i + j$	8192-word uniform leaf
Vector A	$0 \le i < \mathrm{uint}(k)$	seed offset 0; base $256i$; nonce $256\,\mathrm{uint}(k) + \mathrm{uint}(m) + i$	2048-word uniform leaf
Eta vector	$0 \le i < \mathrm{uint}(count)$	seed offset 32; base $256i$; nonce $start + i$ in `W64`	2048-word eta leaf

Table 1: The schedule expressed by `CallerSpec`. Every integer expression is encoded with `W64.of_int` before the leaf call.

For arbitrary caller parameters, these base and nonce equalities are `W64` equalities: the authored integer formulas are encoded modulo 2^{64} by `W64.of_int`. Interpreting them as unbounded natural-number equalities requires separate no-wrap hypotheses; the three equivalence statements do not add those hypotheses.

3 How to Read the Relational Judgments

Each main theorem has the following shape:

```
equiv [GeneratedCaller ~ CallerSpec.reference_caller :
  ={inputs} ==> ={res}].
```

This is a two-program EasyCrypt judgment. Program state with suffix `1` belongs to the generated caller on the left; state with suffix `2` belongs to the authored caller on the right. The notation `={x,y}` requires the listed variables to be equal in both memories. Accordingly:

- the precondition requires equal caller inputs;
- the proof couples corresponding loop iterations;
- the postcondition requires only equal return values; and
- no distributional or functional postcondition for a leaf appears.

The proof scripts use a small collection of tactics:

Tactic	Role in these proofs
`proc`	Exposes the two procedure bodies.
`while(I)`	Couples a pair of loops using relational invariant I.
`callL`	Replaces paired leaf calls with self-equivalence lemma L.
`wp`	Propagates obligations backward through deterministic assignments.
`skip`	Closes the remaining straight-line relational goal after assignments have been processed.
`rewrite/smt`	Rewrites word-level definitions and discharges the integer/word recurrence and guard obligations.

4 The Six Lemmas in the Refinement File

4.1 Three leaf self-equivalences

Lines 11–27 state one self-equivalence for each extracted leaf:

- `uniform8192_leaf_self_equiv`;
- `uniform2048_leaf_self_equiv`;
- `eta2048_leaf_self_equiv`.

Every statement compares an extracted procedure with itself:

```
equiv [ExtractedLeaf ~ ExtractedLeaf : ={arg} ==> ={res}].
proof. by sim. qed.
```

Because the arguments are equal and both sides execute identical code, `sim` proves equal results. These lemmas let the main proofs treat the leaf body as an opaque common component.

Critical boundary. Self-equivalence does not say that a leaf implements SHAKE128, SHAKE256, rejection sampling, centered-trit sampling, uniformity, independence, or even a separately authored functional specification. It says only that the same deterministic extracted procedure returns the same result from equal arguments.

4.2 Three caller-orchestration equivalences

Refinement lemma	Generated procedure	Authored procedure
`expand_vecA_relative_orchestration_equiv`	`_kp_polyveck_expand_vecA`	`CallerSpec.expand_vecA`
`expand_eta_relative_orchestration_equiv`	`_kp_polyvec_expand_eta`	`CallerSpec.expand_eta`
`expand_matA_relative_orchestration_equiv`	`_kp_polymatkm_expand_matA`	`CallerSpec.expand_matA`

All three theorems have equality-only preconditions and equality of the result as postcondition. Their substantive content is in the loop invariants.

5 Proof Walkthrough by Caller

5.1 Vector expansion

The vector proof occupies lines 29–55. Its invariant says, in mathematical notation,

$$\begin{aligned} vp_1 = vp_2, \quad seedp_1 = seedp_2, \\ k_1 = k_2, \quad m_1 = m_2, \\ seedoff_1 = \mathrm{W64.of\_int}(0), \quad i_1 = \mathrm{W64.of\_int}(i_2), \\ base_1 = \mathrm{linear\_base\_word}(i_2), \\ nonce_1 = \mathrm{vector\_nonce\_word} \\ (\mathrm{uint}(k_2), \mathrm{uint}(m_2), i_2), \\ 0 \le i_2 \le \mathrm{uint}(k_2). \end{aligned}$$

This invariant is sufficient for four reasons.

1. `w64_counter_guard` makes the target's unsigned guard `i<k` equivalent to the authored guard `i<W64.to_uintk` while the bound invariant holds.
2. Equality of `vp`, `seedp`, `seedoff`, `base`, and `nonce` gives equal arguments to the 2048-word uniform leaf. The proof then calls `uniform2048_leaf_self_equiv`.

3. `linear_base_word_next`, `vector_nonce_word_next`, and `w64_counter_next` show that the target's increments by 256, 1, and 1 re-establish the closed-form schedule for the next integer counter.

4. `vector_nonce_word_zero_words` proves that the target initializer $(k \ll 8) + m$ equals the authored formula at $i = 0$.

The extracted Jasmin model also contains speculative-load-hardening helpers. The imported model defines `SLH64.protect_64(x,msf)=x`; line 46 unfolds that definition so it cannot obscure the recurrence. This model-level identity does not constitute a constant-time or speculative-execution security proof. The extraction also erases `spill`, `unspill`, and `declassify` calls, so this refinement must not be used as evidence about those source-level operations.

5.2 Eta expansion

The eta proof occupies lines 57–84. The authored procedure saves the input nonce in `start`, whereas the generated procedure increments its local `nonce`. The invariant captures both views:

$$\begin{aligned} vp_1 = vp_2, \quad seedp_1 = seedp_2, \\ count_1 = count_2, \quad start_2 = nonce_2, \\ seedoff_1 = \mathrm{W64.of\_int}(32), \quad i_1 = \mathrm{W64.of\_int}(i_2), \\ base_1 = \mathrm{linear\_base\_word}(i_2), \\ nonce_1 = \mathrm{eta\_nonce\_word}(start_2, i_2), \\ 0 \le i_2 \le \mathrm{uint}(count_2). \end{aligned}$$

The paired call is discharged with `eta2048_leaf_self_equiv`. The lemmas `linear_base_word_next`, `eta_nonce_word_next`, and `w64_counter_next` preserve the invariant. Initialization uses `linear_base_word_zero` and `eta_nonce_word_zero`; guard agreement again comes from `w64_counter_guard`.

Notice what is intentionally absent: the invariant does not assign a distribution to the eta leaf and does not connect `start` to the full key-generation retry counter. It verifies only the parameterized caller.

5.3 Matrix expansion

The matrix proof occupies lines 86–124 and couples nested loops. Its outer invariant is

$$\begin{aligned} matp_1 = matp_2, \quad seedp_1 = seedp_2, \\ rows_1 = rows_2, \quad cols_1 = cols_2, \\ seedoff_1 = \mathrm{W64.of\_int}(0), \quad i_1 = \mathrm{W64.of\_int}(i_2), \\ rowbase_1 = \mathrm{W64.of\_int}(i_2 \,\mathrm{uint}(cols_2)), \\ 0 \le i_2 \le \mathrm{uint}(rows_2). \end{aligned}$$

The inner invariant retains those relations and adds

$$j_1 = \mathrm{W64.of\_int}(j_2), \qquad 0 \le i_2 < \mathrm{uint}(rows_2), \qquad 0 \le j_2 \le \mathrm{uint}(cols_2).$$

Before each leaf call, the generated body computes

$$nonce = (i \ll 8) + j, qquadbase = (rowbase + j) \cdot 256.$$

The authored body supplies `matrix_nonce_word(i,j)` and `matrix_base_word(cols,i,j)`. The lemmas `matrix_nonce_wordE` and `matrix_base_wordE` identify these two forms, after

which `uniform8192_leaf_self_equiv` discharges the call. The inner-loop increment follows from `w64_counter_next`; after the inner loop terminates, `rowbase_word_next` relates `rowbase+cols` to the next row's closed form. Both guards use `w64_counter_guard`.

This proof verifies the nested enumeration and arguments for arbitrary typed `rows` and `cols`. It does not attach a precondition such as $rows \cdot cols \leq 32$ and does not state an array-frame or pointer-separation postcondition.

6 Which Arithmetic Lemmas the Refinement Actually Uses

`easycrypt/spec/KeygenSamplerCallersSpec.ec` contains more arithmetic than `easycrypt/refinement/TargetKeygenSamplerCallers.ec` consumes. The distinction prevents the presence of a nearby lemma from being mistaken for a composed end-to-end theorem.

Proof site	Lemmas explicitly used
All callers	`w64_counter_guard`, `w64_counter_next`, and `W64.to_uint_cmp`.
Vector caller	`linear_base_word_next`, `vector_nonce_word_next`, `vector_nonce_word_zero_words`.
Eta caller	`linear_base_word_next`, `linear_base_word_zero`, `eta_nonce_word_next`, `eta_nonce_word_zero`.
Matrix caller	`matrix_nonce_wordE`, `matrix_base_wordE`, `rowbase_word_next`, and unfolding `uniform_seed_offset_i`.

The specification separately proves capacity bounds, disjoint integer regions, low-16-bit nonce facts, and concrete mode-2 schedules. Those lemmas compile in the same theory, but none is cited by the three relational proof scripts. Their status is therefore:

- **machine-checked arithmetic facts** in `easycrypt/spec/KeygenSamplerCallersSpec.ec`;
- **useful premises** for future memory-frame and full key-generation composition proofs; but
- **not yet connected** to a theorem about `_keypair_full_m23`.

7 The Adjacent Mode-2 Facts

For $k = 2$ and $m = 3$, the companion specification proves concrete matrix and vector schedules plus eta retry-nonce arithmetic:

Role	Bases	Nonces
2×3 matrix	$0, 256, 512, 768, 1024, 1280$	$0, 1, 2, 256, 257, 258$
Length-2 vector	$0, 256$	$515, 516$
Retry-r eta nonces	No paired-call base schedule is proved	$5r, 5r+1, 5r+2, 5r+3, 5r+4$

The first two rows are direct concrete schedule lemmas. For eta, the theorem `mode2_eta_retry_schedule` proves only the five nonce formulas. Each individual `expand_eta` caller uses base $256i$, but grouping the first three and last two nonce slots into paired count-3/count-2 calls is intended composition, not a theorem in the current artifact.

The specification also proves generic capacity facts under explicit hypotheses $rows \cdot cols \leq 32$ and $count \leq 8$, plus disjointness of 256-word integer regions. These results describe the intended mode-2 schedule. The current refinement does not prove that the omitted full key-generation procedure supplies these parameters, shares the intended seed buffer, makes the paired eta calls, or advances its retry counter as modeled.

8 Exact Claim Boundary

8.1 What is established

Subject to the checked artifact and toolchain recorded in the repository, the formal relational judgments establish equality of each returned array from equal typed inputs. The checked proof scripts derive those judgments by:

- lockstep agreement of generated word counters with authored integer counters;
- equal seed offsets, output bases, and nonces at every paired leaf call; and
- coupled nested-matrix and linear vector/eta iterations.

These are invariant and proof-structure facts used to establish result equality. They are not a separately exported call-trace theorem. The repository verification gate additionally establishes:

- source-to-extraction reproducibility for the generated caller surface; and
- successful `-no-eco` compilation of all three manifest entries, with no authored `admit`/`abort` commands or axiom declarations.

The proof-hole and axiom scans cover the two authored files, not the complete imported dependency closure. Likewise, extraction drift checks agreement with the current `jasmin2ec` output; it does not prove the extractor or imported EasyCrypt/Jasmin models correct.

8.2 What remains open

The result does **not** prove any of the following:

- SHAKE128 or SHAKE256 correctness, sampler functional correctness, uniformity, centered-trit distribution, independence, or rejection-loop termination/losslessness;
- preservation of unused array regions, source-level output frames, pointer separation, or pointer memory safety;
- the identity of seed-buffer slices or their relation to `_kp_expand_seedbuf` and the paper's named seeds;
- reachability from or parameter instantiation by `_keypair_full_m23`, including paired eta calls and retry advancement;
- singular-value rejection or the later `egen-b0` adjustment; or
- end-to-end correspondence with paper `expandA`/`expandS`, the public key-generation API, or a security-game distribution.

Consequently, the parameterized caller schedule is VERIFIED, while the broader mode-2 sampler composition remains PARTIAL in the repository traceability manifest.

9 Reproducing the Evidence

From `haetae-ref-easycrypt/`, run:

```
./scripts/verify-keygen-sampler-callers-proof.sh
```

The script performs these checks in order:

1. starts a dedicated Why3 server;

2. checks all pinned source hashes;
3. regenerates the sampler-caller extraction and rejects file, procedure, or byte-level drift;
4. compiles the generated target, authored specification, and authored refinement with `easycryptcompile-no-eco`;
5. rejects `admit` or `abort` proof commands in the authored files; and
6. rejects authored axiom declarations and records artifact hashes.

The retained run in `logs/keygen-sampler-callers-proof-summary.txt` reports 62 pinned inputs, zero drift across 23 generated theories and the exact 29-procedure inventory, 3/3 manifest entries compiled, no proof holes, no axiom declarations, and `RESULT:PASS`.

Build this guide independently with:

```
cd latex
make
```

The resulting PDF is `latex/build/TargetKeygenSamplerCallers.pdf`. The LaTeX build does not replay the EasyCrypt proof; the two commands serve separate purposes.

10 Source Index and Fingerprints

Artifact	Relevant lines	Content
Refinement	11–27	Three leaf self-equivalences.
Refinement	29–55	Vector caller equivalence.
Refinement	57–84	Eta caller equivalence.
Refinement	86–124	Matrix caller equivalence.
Specification	10–143	Schedule definitions and word-encoding lemmas.
Specification	145–319	Capacity, disjointness, low-16-bit, and mode-2 facts.
Specification	321–370	The authored `CallerSpec` procedures.
Generated target	1276–1394	The three generated caller procedures.

The source hashes checked against the retained passing report were:

```
ab3a8dd3695f6204bd93c17c033faa40ab18a34dee2db6cfdd78c716662ac66f  KeygenSamplerCallersTarget.ec
943400204fff763f3d099459a53785be1911bcd848dad0ac57ae0554d09a07fd  KeygenSamplerCallersSpec.ec
5228ffd0fd0507b5dd749c0186e5965b62a55eafd40c1650ab770f1054122521  TargetKeygenSamplerCallers.ec
```

If any fingerprint changes, rerun the proof gate and review this guide against the new source before treating the explanation as current.

11 Bottom Line

`easycrypt/refinement/TargetKeygenSamplerCallers.ec` is a clean bridge between generated caller control flow and an auditable integer schedule. Its central technique is lockstep relational reasoning: preserve a word/integer loop invariant, pass equal arguments to an identical extracted leaf, and use small arithmetic lemmas to re-establish the invariant. This is meaningful progress because it isolates and verifies the caller orchestration. Its deliberate use of leaf self-equivalence is also the reason the theorem must not be presented as sampler correctness or complete mode-2 key-generation refinement.